TCC Seguridad Informática 2026-2

Endurecimiento de una aplicación de gestión documental: auditoría, explotación y mitigación con evidencia reproducible

Trabajo Colaborativo Contextualizado — Seguridad Informática (2026-2) Universidad de Córdoba · Facultad de Ingeniería · Ingeniería de Sistemas

Integrantes de la CIPA

Integrante	Rol en el trabajo
[Nombre 1]	Líder de proyecto y análisis de riesgos (MAGERIT, arquitectura, STRIDE, coordinación)
[Nombre 2]	Desarrollo y endurecimiento (aplicación Rust, criptografía, sesiones, segundo factor)
[Nombre 3]	Pentesting y evidencias (suite de ataques, capturas, panel interactivo, sustentación)

Fecha de entrega: 20 de noviembre de 2026 · **Sustentación:** 21 de noviembre de 2026

Resumen ejecutivo

Este trabajo construye una aplicación web de gestión documental —**MiniGestor Documental**— implementada en Rust y ejecutada íntegramente en Docker, con **dos modos del mismo binario**: uno deliberadamente vulnerable y otro endurecido. El modo vulnerable materializa diez defectos de seguridad reales; el modo endurecido aplica las contramedidas correspondientes. Sobre ambos se ejecuta una **suite de nueve ataques automatizados** que actúa como especificación ejecutable: cada ataque **debe** funcionar contra la versión vulnerable y **debe** quedar bloqueado en la version endurecida.

Resultado verificado: **18 de 18 expectativas cumplidas** (9 ataques × 2 modos), sin fallos ni pruebas omitidas. La evidencia completa —respuestas HTTP, salidas de prueba, capturas de tráfico y un panel interactivo que las presenta— se regenera con un único comando.

Los enfoques del curso cubiertos son los cuatro: **OWASP Top 10**, **criptografía aplicada**, **análisis de riesgos MAGERIT v3** y **detección/simulación de ataques de red**. El aporte metodológico central es el uso de una **doble versión como experimento controlado**: la misma aplicación, el mismo esquema de datos y las mismas pruebas, cambiando únicamente las decisiones de seguridad. Así, la afirmación «el control funciona» deja de ser una opinión y se convierte en un resultado reproducible.

---

1. Introducción

La seguridad informática suele enseñarse como un conjunto de conceptos separados: listas de vulnerabilidades, algoritmos criptográficos, marcos de gestión de riesgos y herramientas de pentesting. En la práctica profesional esos conceptos operan juntos: un defecto de validación de entrada se convierte en fuga de datos, la fuga permite romper un esquema de hashing débil, y el esquema de hashing débil anula cualquier control perimetral. Este trabajo integra los cuatro enfoques del curso sobre **un solo sistema** para mostrar esas cadenas de dependencia.

El proyecto parte de una pregunta concreta y verificable: *¿qué cambia, exactamente, entre una aplicación que un atacante compromete en minutos y la misma aplicación con controles aplicados?* Para responderla no se compara documentación contra documentación, sino **ejecución contra ejecución**: los mismos nueve ataques, automatizados, contra las dos versiones.

2. Planteamiento del problema

2.1 Contexto

Las organizaciones que digitalizan procesos administrativos acumulan documentos con información sensible —contratos, historias clínicas, datos financieros— en aplicaciones web construidas con plazos cortos y sin revisión de seguridad. El patrón de fallo es conocido: autenticación con hashes obsoletos, autorización verificada solo en la interfaz, consultas SQL construidas por concatenación, archivos almacenados en claro y ausencia de trazabilidad. Cada uno de esos defectos, por separado, parece menor; en conjunto producen brechas con impacto regulatorio.

2.2 Problema

Una aplicación de gestión documental sin controles de seguridad permite a un atacante con conocimientos básicos leer documentos de otros usuarios, recuperar credenciales, escalar privilegios y escribir archivos arbitrarios en el servidor, sin dejar rastro auditable.

2.3 Justificación

- **Pertinencia académica:** cubre los cuatro enfoques del curso en un único artefacto verificable, en lugar de cuatro actividades inconexas.
- **Pertinencia profesional:** reproduce el ciclo real de trabajo (análisis de riesgos → prueba de penetración → remediación → re-verificación), que es el ciclo que exigen marcos como ISO/IEC 27001 y OWASP ASVS.
- **Valor didáctico:** el modo vulnerable es un material de estudio; el modo endurecido permite contrastar cada mitigación con su defecto, en el mismo código.
- **Sostenibilidad:** todo el laboratorio se levanta con un comando y no depende de máquinas virtuales ni de herramientas instaladas en el equipo, lo que permite que cualquier integrante —o el docente— reproduzca los resultados.

3. Objetivos

3.1 Objetivo general

Implementar mecanismos de seguridad informática en una aplicación web de gestión documental, mediante análisis de riesgos, criptografía aplicada, controles del OWASP Top 10 y pruebas de penetración, demostrando el antes y el después con evidencia técnica reproducible.

3.2 Objetivos específicos

1. Diseñar la arquitectura del laboratorio y de la aplicación con dos modos de operación sobre un mismo código.
2. Identificar activos, amenazas y riesgos aplicando MAGERIT v3 en escala simplificada, con trazabilidad a controles concretos.
3. Implementar la versión vulnerable con diez defectos representativos de A01, A02, A03, A04, A05, A07 y A09 del OWASP Top 10 2021.

4. Implementar la versión endurecida con Argon2id, AES-256-GCM, sesiones firmadas con estado en servidor, TOTP, control de acceso verificado en servidor, validación de entradas, cabeceras de seguridad y auditoría.
5. Construir una suite de ataques automatizada que verifique, para cada defecto, su explotación en la versión vulnerable y su bloqueo en la versión endurecida.
6. Producir evidencia verificable: respuestas HTTP, capturas de tráfico, resultados consolidados y un panel interactivo de consulta.
7. Documentar el proceso, las pruebas y los resultados de forma que un tercero pueda reproducirlos.

3.3 Alcance y limitaciones

Incluido: aplicación web con autenticación, autorización, carga, búsqueda, lectura y descarga de documentos, panel administrativo y auditoría; laboratorio Docker con base de datos, gateway TLS, red aislada y herramientas de captura; los cuatro enfoques del curso; suite de ataques y panel de evidencias.

Excluido o limitado, de forma explícita:

- No se realizan pruebas contra sistemas reales de la universidad ni de terceros. Todo el tráfico se genera dentro de la red del laboratorio.
- El análisis MAGERIT es una aplicación metodológica en escala 1–5, no una entrega completa del método (no incluye análisis de dependencias entre activos ni valoración económica).
- El firewall (nftables) se documenta como política de referencia ejecutada en un contenedor propio; no sustituye un cortafuegos perimetral real.
- La aplicación no implementa firma digital de documentos, antivirus de archivos cargados ni federación de identidad: son trabajo futuro declarado.
- Siete de los nueve ataques se ejecutan desde el panel de evidencias; los dos que requieren privilegios del host (path traversal y captura de tráfico) se ejecutan con el comando que el panel indica.
- Los secretos incluidos (clave maestra, secreto de sesión, contraseñas demo) son **valores de laboratorio** y su exposición forma parte del ejercicio de auditoría.

4. Marco teórico

4.1 OWASP Top 10 (2021)

Catálogo de las categorías de riesgo más relevantes en aplicaciones web. Este trabajo aborda: **A01** control de acceso quebrado, **A02** fallas criptográficas, **A03** inyección, **A04** diseño inseguro, **A05** configuración incorrecta, **A07** fallas de identificación y autenticación y **A09** fallas de registro y monitoreo.

4.2 MAGERIT v3

Metodología española de gestión de riesgos de sistemas de información. Modela **activos** (clasificados en dimensiones de confidencialidad, integridad, disponibilidad, autenticidad y trazabilidad), **amenazas** con una frecuencia estimada y un **impacto** expresado como degradación del activo. El riesgo se aproxima como `riesgo = frecuencia × degradación`. Sobre ese valor se seleccionan **salvaguardas** y se evalúa el **riesgo residual**.

4.3 Criptografía aplicada

- **Funciones de derivación de claves (KDF) frente a funciones de hash simples.** MD5 fue diseñado para velocidad, propiedad exactamente opuesta a la deseable para almacenar contraseñas. Argon2id (RFC 9106), ganador del Password Hashing Competition, incorpora costo de memoria, de tiempo y de paralelismo, además de una sal única por usuario.

- **Cifrado autenticado (AEAD).** AES-256-GCM garantiza confidencialidad e integridad en una sola operación; exige un nonce distinto por mensaje, condición que este trabajo cumple generando 12 bytes aleatorios por documento.
- **Integridad de sesión.** HMAC-SHA256 sobre el identificador de sesión permite detectar cookies manipuladas, con comparación en tiempo constante para no filtrar información por temporización.
- **Segundo factor TOTP (RFC 6238).** Código de 6 dígitos derivado de HMAC-SHA1 sobre un contador de 30 segundos, con ventana de tolerancia de ±1 paso.
- **Transporte.** TLS 1.2/1.3 con cifrados AEAD y HSTS; cookies `Secure` para que el navegador no las envíe por canal no cifrado.

4.4 STRIDE

Modelo de amenazas de Microsoft que clasifica las amenazas en **S**uplantación, **T**ampering, **R**epudio, **I**nformación expuesta, **D**enegación de servicio y **E**levación de privilegios. Se aplica sobre el diagrama de flujo de datos para ubicar cada amenaza en un flujo concreto.

4.5 Defensa en profundidad

Principio de superponer controles independientes para el mismo riesgo, de modo que el fallo de uno no implique el compromiso total. En este trabajo se aplica de forma verificable: el límite de intentos de autenticación existe en la aplicación **y** en el gateway; el escape de salida convive con una política CSP; el cifrado en reposo convive con el cifrado en tránsito.

5. Metodología

5.1 Diseño del experimento controlado

La decisión metodológica central es **no comparar dos proyectos**, sino **un proyecto con dos políticas**. La aplicación es un único binario cuyo comportamiento depende de la variable de entorno `MODE` (`vulnerable` o `hardened`). Comparten esquema de base de datos, rutas, vistas y flujo; difieren en las decisiones de seguridad. Cada instancia usa además una base de datos independiente (`minigestor_vuln` y `minigestor_hard`), de modo que las evidencias del «después» no se contaminan con el estado del «antes».

5.2 La suite de ataques como especificación ejecutable

Cada ataque es un script con un contrato explícito:

```
attacks/NN-ataque.sh <url-objetivo> <vulnerable|hardened>
  exit 0  → el ataque tuvo éxito  (el defecto está presente)
  exit 1  → el ataque fue bloqueado (la mitigación funciona)
  exit 2  → la prueba no pudo ejecutarse
```

`attacks/run-all.sh` ejecuta los nueve ataques contra ambos modos, compara cada resultado con la expectativa (`vulnerable → exit 0`, `hardened → exit 1`) y produce `evidence/results.json`. **El informe no puede afirmar que una mitigación funciona si la suite no lo demuestra**: si una expectativa falla, el script termina con código de error y el panel muestra el fallo en rojo.

Esta elección convierte la verificación en algo repetible y resistente al autoengaño: no se revisa a ojo, se ejecuta.

5.3 Fases

1. **Análisis** — inventario de activos, identificación de amenazas (STRIDE) y valoración de riesgos (MAGERIT).
2. **Construcción** — implementación de la versión vulnerable con los defectos trazados a riesgos.
3. **Explotación** — ejecución de la suite contra la versión vulnerable (estado «rojo»).
4. **Remediación** — implementación de los controles en la versión endurecida.
5. **Re-verificación** — misma suite contra la versión endurecida (estado «verde») y análisis del riesgo residual.
6. **Documentación y evidencia** — consolidación de resultados y panel interactivo.

5.4 Ética y legalidad

Todo el tráfico se genera dentro de una red Docker aislada y los servicios se publican exclusivamente en `127.0.0.1`. Los datos son simulados. No se ejecuta ninguna prueba contra infraestructura de la universidad ni de terceros; hacerlo constituiría un delito conforme a la Ley 1273 de 2009 (Colombia), que tipifica el acceso abusivo a sistemas informáticos.

6. Diseño de la solución

6.1 Arquitectura del laboratorio

La arquitectura completa se documenta en `docs/diagramas/arquitectura.svg`. Sus decisiones principales:

Componente	Función	Decisión de seguridad
`app-vuln` (:18080)	Aplicación con defectos sembrados	Publicada solo en `127.0.0.1`, con banner de advertencia permanente
`gateway` (:18443 TLS, :18083 → 301)	Terminación TLS, HSTS, límite de tasa	TLS 1.2/1.3, solo cifrados AEAD, `limit_req` en `/login`
`app-hard` (:18091, tras el gateway)	Aplicación endurecida	No publicada a Internet; en producción estaría solo en la red interna
`db` (sin puerto)	PostgreSQL 16 con dos bases	Sin puerto publicado: solo alcanzable dentro de `lab_net`
`attacker` (perfil `tools`)	`tcpdump` y utilidades	`NET_ADMIN`/`NET_RAW` limitados al contenedor
`evidence` (:18090)	Panel interactivo de evidencias	Solo lectura del repositorio (`:ro`)
`lab_net` 172.28.60.0/24	Red del laboratorio	Aísla el tráfico de las pruebas

6.2 Flujo de datos y amenazas (STRIDE)

El diagrama `docs/diagramas/dfd-stride.svg` identifica nueve flujos y las amenazas asociadas:

Flujo	Amenaza (STRIDE)	Defecto en la versión vulnerable	Ataque que la demuestra
F1 credenciales	Suplantación	Sin límite de intentos; mensajes que enumeran cuentas	`04-bruteforce`
F2 hash → D1	Información expuesta	MD5 sin sal	`06-weak-hash`
F3 término de búsqueda	Tampering	SQL concatenado	`01-sqli-search`
F4 SQL → D2	Elevación de privilegios	Sin filtro por propietario	`01-sqli-search`

Flujo	Amenaza (STRIDE)	Defecto en la versión vulnerable	Ataque que la demuestra
F5 archivo + nombre	Tampering	Nombre del cliente usado como ruta	`07-upload-traversal`
F6 escritura	Información expuesta	Contenido en claro, nombre predecible	`07-upload-traversal`
F7 contenido + HTML	Elevación (cliente)	Salida sin escapar; cookie sin `HttpOnly`	`02-xss-stored`
F8 evento	Repudio	Sin tabla de auditoría	`08-headers` (ausencia de controles)
F9 descarga	Información expuesta	Sin verificación de propiedad	`03-idor`

6.3 Análisis de riesgos (MAGERIT v3, escala 1–5)

El inventario completo está en `docs/riesgos/magerit.yml`. Se identificaron 6 activos y 12 riesgos; el mapa de calor se publica en el panel de evidencias.

Activos

ID	Activo	Tipo	Criticidad
A1	Servicio web de gestión documental	[S] servicio	Alta
A2	Base de datos (usuarios, metadatos, auditoría)	[D] datos	Alta
A3	Documentos almacenados	[I] información	Alta
A4	Credenciales, secretos de sesión y clave maestra	[K] claves	Alta
A5	Red del laboratorio y gateway TLS	[COM] comunicaciones	Media
A6	Código fuente y artefactos	[SW] software	Media

Riesgos principales (`R = F × D`; nivel: Crítico ≥16, Alto 9–15, Medio 4–8, Bajo ≤3)

ID	Activo	Amenaza	F	D	R	Nivel	Salvaguarda aplicada	Residual
R1	A2	Inyección SQL en la búsqueda	5	5	25	Crítico	Consulta parametrizada	Bajo
R2	A3	Lectura de documentos ajenos (IDOR)	5	5	25	Crítico	Verificación de propiedad en servidor	Bajo
R3	A1	XSS almacenado y reflejado	4	4	16	Crítico	Escape de salida + CSP + cookie `HttpOnly`	Bajo
R4	A2	Recuperación de contraseñas por hash débil	5	5	25	Crítico	Argon2id con sal única	Medio
R5	A4	Falsificación de sesión y escalada	5	5	25	Crítico	Sesión en servidor + HMAC-SHA256 + rol revalidado	Bajo
R6	A1	Fuerza bruta contra el login	5	4	20	Crítico	5 intentos/min + bloqueo 5 min + TOTP + `limit_req`	Bajo
R7	A3	Documentos expuestos en reposo	4	5	20	Crítico	AES-256-GCM con nonce aleatorio	Bajo
R8	A3	Escritura arbitraria de archivos	4	4	16	Crítico	Lista blanca + 5 MiB + nombre opaco	Bajo
R9	A4	Secretos expuestos por `/debug/config`	3	5	15	Alto	El endpoint no existe en el binario endurecido	Bajo
R10	A5	Interceptación de credenciales (MITM)	4	5	20	Crítico	TLS + HSTS + `Secure`	Bajo
R11	A1	Falta de trazabilidad	4	3	12	Alto	Tabla `audit_log` + panel administrativo	Medio

ID	Activo	Amenaza	F	D	R	Nivel	Salvaguarda aplicada	Residual
R12	A6	Configuración insegura del navegador	3	3	9	Alto	CSP, XFO, `nosniff`, Referrer-Policy	Bajo

Riesgos residuales que se asumen explícitamente: R4 y R11 quedan en nivel medio porque un hash Argon2id sigue siendo crackeable con diccionario si la contraseña es débil (la mitigación real es la política de contraseñas y el segundo factor), y porque la auditoría local no sustituye un SIEM centralizado con retención y alertas.

6.4 Controles y contramedidas

Riesgo	Control implementado	Ubicación en el código
R1	Consultas parametrizadas con enlace de parámetros	`app/src/routes/docs_routes.rs`
R2	Verificación de propiedad o rol en cada acceso + auditoría del intento	`app/src/routes/docs_routes.rs`
R3	Escape de salida centralizado, CSP sin scripts, cookie `HttpOnly`	`app/src/views.rs`, `app/src/security.rs`
R4	Argon2id (19 MiB, t=2, p=1) con sal por usuario; verificación de hashes legados para migración	`app/src/auth.rs`
R5	Identificador de sesión de 32 bytes aleatorios + HMAC-SHA256 + tabla `sessions` con expiración	`app/src/session.rs`
R6	Ventana de 5 intentos/minuto, bloqueo temporal de la cuenta, respuesta genérica, TOTP, `limit_req`	`app/src/security.rs`, `app/src/routes/auth_routes.rs`, `lab/gateway/nginx.conf`
R7	AES-256-GCM con nonce de 12 bytes por documento	`app/src/crypto.rs`
R8	Lista blanca de extensiones, límite de tamaño, nombre opaco (UUID)	`app/src/routes/docs_routes.rs`
R9	El endpoint de depuración solo se registra en modo vulnerable	`app/src/routes/mod.rs`
R10	TLS 1.2/1.3, HSTS, redirección 301, cookies `Secure`	`lab/gateway/nginx.conf`, `app/src/session.rs`
R11	`audit_log` con actor, acción, objetivo y resultado; panel de auditoría	`app/src/db.rs`, `app/src/routes/admin_routes.rs`
R12	Cabeceras de endurecimiento en un middleware	`app/src/security.rs`

7. Implementación

7.1 Tecnologías y decisiones

Capa	Tecnología	Justificación
Lenguaje	Rust (edición 2021)	Seguridad de memoria sin recolector; ecosistema criptográfico de referencia (`argon2`, `aes-gcm`, `hmac`)
Servidor HTTP	`axum` 0.8 sobre `tokio`	Extracción de tipos explícita, composición por middleware
Acceso a datos	`sqlx` 0.8 (consultas en tiempo de ejecución)	Permite mostrar la diferencia entre SQL concatenado y parametrizado en el mismo archivo

Capa	Tecnología	Justificación
Base de datos	PostgreSQL 16	Restricciones, tipos y funciones de fecha; dos bases independientes
Contenedores	Docker 29.9 + Compose 5.6	Reproducibilidad sin instalar herramientas en el equipo
Gateway	nginx 1.27	Terminación TLS, HSTS y límite de tasa independiente
Panel	Sitio estático generado desde el repositorio	Cero acoplamiento: el panel lee las evidencias, no las produce

Tamaño del artefacto: **2.174 líneas** propias de Rust y HTML en `app/`, y **1.334 líneas** de shell, Python y HTML en `attacks/`, `site/` y el análisis de riesgos; **241 dependencias** fijadas en `Cargo.lock` para garantizar compilaciones reproducibles.

7.2 La aplicación en dos modos

El punto de decisión está centralizado. Un ejemplo real del contraste, en el mismo archivo y a pocas líneas de distancia:

```
// v1 — DEFECTO A03: el término se concatena dentro del SQL.
let sql = format!(
    "SELECT ... FROM documents d JOIN users u ON u.id = d.owner_id \
     WHERE d.title LIKE '%{}%' ORDER BY d.id", term
);
sqlx::query(&sql).fetch_all(&state.pool).await?

// v2 — consulta parametrizada: el payload es texto, nunca sintaxis.
sqlx::query("SELECT ... WHERE d.title ILIKE $1 AND (d.owner_id = $2 OR $3) ...")
    .bind(format!("%{}%", term)).bind(user.id).bind(user.is_admin())
    .fetch_all(&state.pool).await?
```

El mismo patrón se repite en el almacenamiento de contraseñas (`md5_hex` frente a Argon2id), en el escape de salida (`interp` devuelve el valor crudo o escapado), en la sesión (cookie `usuario:rol` frente a identificador opaco firmado con HMAC) y en el cifrado de documentos (escritura directa frente a `crypto::seal` con AES-256-GCM).

7.3 Detalles de implementación relevantes

- **Cifrado en reposo.** Formato `[nonce 12 B][ciphertext + tag GCM]`. El nonce aleatorio por documento evita la reutilización, que en GCM es catastrófica.
- **Migración de hashes.** `verify_password` reconoce el formato PHC de Argon2id y, si el valor no lo es, cae al hash legado MD5. Esto permite migrar usuarios sin invalidar cuentas, un requisito real de cualquier plan de remediación.
- **Segundo factor.** TOTP implementado sobre HMAC-SHA1 con truncamiento dinámico (RFC 4226 §5.3), secreto de 160 bits codificado en base32 (RFC 4648) y ventana de ±1 paso de 30 segundos.
- **Autorización.** En el modo endurecido el rol nunca se toma de la cookie: se consulta nuevamente en la base de datos en cada operación administrativa, de modo que una cookie manipulada no otorga privilegios.
- **Auditoría.** Cada lectura, descarga, carga, inicio de sesión y acceso denegado genera un registro con actor, acción, objetivo y resultado.

8. Pruebas y resultados

8.1 Matriz antes / después

Ejecución verificada el 6 de noviembre de 2026 (`bash attacks/run-all.sh`, salida consolidada en `evidence/results.json`):

#	Ataque	OWASP	Vulnerable	Endurecido
01	SQL injection en la búsqueda (UNION SELECT)	A03	**Éxito** (hashes expuestos, HTTP 200)	**Bloqueado** (0 resultados, sin error)
02	XSS almacenado en el título	A03	**Éxito** (`<script>` renderizado)	**Bloqueado** (`<script>`)
03	IDOR en `/docs/{id}`	A01	**Éxito** (documento confidencial ajeno, HTTP 200)	**Bloqueado** (HTTP 403)
04	Fuerza bruta en `/login`	A07	**Éxito** (8 intentos sin límite)	**Bloqueado** (HTTP 429 al 6.º intento)
05	Falsificación de cookie (`sid=alice:admin`)	A01	**Éxito** (panel de administración)	**Bloqueado** (firma inválida → redirección)
06	Hashes débiles (diccionario mínimo)	A02	**Éxito** (contraseñas recuperadas)	**Bloqueado** (Argon2id, sin coincidencias)
07	Path traversal en la carga	A04	**Éxito** (archivo escrito en `/tmp`)	**Bloqueado** (nombre opaco)
08	Cabeceras de seguridad ausentes	A05	**Éxito** (4 de 4 ausentes)	**Bloqueado** (CSP, XFO, nosniff, Referrer-Policy)
09	Credenciales en claro sobre HTTP	A02	**Éxito** (`alice123` legible en el `.pcap`)	**Bloqueado** (solo texto cifrado en TLS)

Resumen: 18 ejecuciones, 18 PASS, 0 FAIL, 0 SKIP.

8.2 Hallazgos destacados

Hallazgo 1 — La inyección SQL no necesita herramientas (R1). El payload `' UNION SELECT 1, username, password_hash, 'x', 0, 'y' FROM users --` sobre `/search` devuelve, en la misma tabla de resultados, los hashes de todos los usuarios. El ataque funciona con `curl` en una sola línea: no hace falta `sqlmap` para demostrarlo, y por eso «nadie nos va a atacar así» es una premisa falsa.

Hallazgo 2 — El hash es el eslabón más débil de la cadena (R4). Los hashes MD5 obtenidos se revierten con un diccionario de diez palabras: las tres cuentas demo se recuperan en menos de un segundo. Con Argon2id (19 MiB, t=2, p=1) el mismo diccionario no produce ninguna coincidencia, y el costo por intento —19 MiB de memoria— es lo que hace inviable el ataque a escala.

Hallazgo 3 — Una cookie en texto plano equivale a no tener autenticación (R5). Enviar `Cookie: sid=alice:admin` abre el panel administrativo sin conocer ninguna contraseña. El defecto no está en la criptografía sino en el diseño: el servidor confía en un valor que el cliente controla.

Hallazgo 4 — La captura de tráfico convierte una sospecha en una prueba (R10). La captura `evidence/network/09-http.pcap` contiene la contraseña `alice123` en texto claro; la captura equivalente sobre TLS no contiene ningún dato legible. Es la evidencia más contundente de por qué el cifrado en tránsito no es opcional.

Hallazgo 5 — La auditoría es un control de seguridad, no un requisito burocrático (R11). Sin `audit_log`, un IDOR exitoso es indistinguible de un acceso legítimo. Con auditoría, el intento de leer un documento ajeno queda registrado con actor, objetivo y resultado: la detección pasa a ser posible.

8.3 La suite detectó dos defectos propios

La primera ejecución completa arrojó **16 de 18 PASS**. Los dos fallos aparecieron en el modo vulnerable, que es donde los ataques *deben* funcionar; es decir, la suite se estaba engañando a sí misma. El análisis produjo dos

correcciones de valor metodológico:

1. **SQL en varias líneas.** La sentencia vulnerable estaba escrita en tres líneas. El `--` del payload comentaba solo el final de su línea y el `ORDER BY` siguiente rompía la consulta: la inyección producía un error 500 en lugar de una fuga. Se reescribió en una sola línea, que es además como la construyen la mayoría de las aplicaciones reales. **Lección:** una prueba que falla puede estar señalando un defecto en la prueba, no en el sistema.
2. **`curl -F` interpreta el carácter `<`.** El título malicioso `<script>…` se interpretó como «leer el valor desde un archivo». La solución fue `--form-string`, que trata el valor como literal.

Ninguna de las dos correcciones tocó el comportamiento de seguridad buscado. El resultado final, tras verificarlas, es 18/18.

8.4 Evidencia disponible

Evidencia	Ruta
Resultado consolidado de la suite	`evidence/results.json`
Salida literal de cada ataque por modo	`evidence/http/<modo>/<ataque>.log`
Respuestas HTTP crudas	`evidence/http/<modo>/*.html`
Capturas de tráfico (HTTP en claro / TLS)	`evidence/network/09-http.pcap`, `09-tls.pcap`
Panel interactivo (consulta y ejecución en vivo)	`http://127.0.0.1:18090`
Diagramas	`docs/diagramas/*.svg`
Inventario de riesgos	`docs/riesgos/magerit.yml`

8.5 Demostración en vivo desde el panel

Para que el docente pueda presenciar las vulnerabilidades en lugar de leer sobre ellas, el panel de evidencias incorpora un **ejecutor de pruebas**: cada fila de la matriz tiene un botón que lanza el ataque real contra el modo que se elija y devuelve la salida del script, el código de salida y la interpretación del resultado.

La consecuencia práctica es que la sustentación no depende de recordar comandos ni de la pericia de quien escribe en la terminal: se hace clic, se ve la salida y se explica. Siete de los nueve ataques se ejecutan así; los otros dos (path traversal y captura de tráfico) requieren privilegios sobre el host —inspeccionar el sistema de archivos del contenedor o capturar paquetes con `NET_ADMIN`— y el panel muestra el comando exacto para correrlos en la terminal, sin ocultar la limitación.

Guardas del ejecutor. Un endpoint que ejecuta procesos es, en sí mismo, una superficie de ataque. La versión final se sometió a una **verificación de seguridad independiente** (revisión read-only del código y de la configuración del contenedor) que confirmó la ausencia de ejecución arbitraria de comandos y de bypass de la lista blanca, y señaló debilidades de aislamiento y de consumo de recursos. Todas se corrigieron antes de la entrega:

Hallazgo de la verificación	Corrección aplicada
El control de `Host` es falsificable: no autentica el origen	Token por proceso (`/api/token` sin CORS + `X-Panel-Token`), que una página de otro origen no puede leer (CSRF / DNS rebinding)
Salida y cuerpo de petición sin límite	Cuerpo acotado a 4 KB (413) y salida leída desde archivo, recortada a 8 KB
El timeout no cerraba los procesos hijos	`start_new_session=True` y `SIGKILL` al grupo de procesos completo

Hallazgo de la verificación	Corrección aplicada
Caché compartido entre ejecuciones	Caché de cookies por ejecución dentro del directorio temporal
El proceso corría como `root`	Usuario sin privilegios (`panel`, uid 10001)
Fuga de rutas internas y tracebacks	Los errores se registran en el contenedor y al cliente solo se devuelve un mensaje genérico

Las restricciones que quedaron implementadas son:

1. Token de sesión por proceso + solo `Host` `127.0.0.1`/`localhost`.
2. Lista blanca de ataques y patrón estricto del identificador; argumentos en lista, sin shell.
3. Modo validado contra un diccionario cerrado, con destinos fijos del laboratorio.
4. Una ejecución simultánea, timeout con muerte del grupo de procesos y salida acotada.
5. Directorio temporal y caché propios de cada ejecución, eliminados al terminar.
6. Contenedor sin privilegios: usuario no root, sistema de archivos de solo lectura, `cap_drop: ALL`, `no-new-privileges`, repositorio en `:ro` y sin *socket* de Docker.

Los ataques en vivo no sobrescriben las evidencias oficiales: escriben en un directorio temporal, de modo que la matriz que se presenta sigue siendo la de la última corrida completa de la suite.

9. Discusión

Lo que se aprendió sobre el orden de las mitigaciones. El análisis de riesgos y la explotación coincidieron en la prioridad: control de acceso (R1, R2, R5) antes que criptografía. Una aplicación con AES-256-GCM pero sin verificación de propiedad sigue entregando documentos ajenos, solo que cifrados en disco. La criptografía protege el activo cuando el control de acceso ya funciona.

Lo que costó más de lo previsto. El segundo factor y el cifrado en reposo fueron mecánicos; lo que consumió tiempo fue la *coherencia*: que cada defecto tuviera su control y su prueba, y que las tres piezas apuntaran al mismo archivo y a la misma línea. Ese trabajo de trazabilidad es el que convierte un ejercicio en un informe defendible.

Sobre el uso de Rust. Rust aporta seguridad de memoria y un ecosistema criptográfico confiable, y el binario es notablemente rápido; sin embargo, **no hace segura a la aplicación**. El defecto más grave del trabajo (confiar en el rol que envía el cliente) es de diseño, y habría sido idéntico en cualquier lenguaje. La elección del lenguaje reduce una clase de fallos (desbordamientos, uso después de liberar) pero no sustituye el análisis.

Límites de la demostración. La suite prueba lo que la suite prueba: los nueve defectos sembrados y sus nueve controles. No certifica la ausencia de otras vulnerabilidades. Un pentest profesional complementaría esto con análisis de dependencias, revisión de configuración del sistema operativo, pruebas de concurrencia y fuzzing de las rutas de carga.

10. Conclusiones

1. Es posible demostrar el efecto de un programa de endurecimiento **con evidencia reproducible** en lugar de afirmaciones: 18 de 18 expectativas cumplidas, regenerables con un comando.
2. Los cuatro enfoques del curso no son temas independientes: forman una cadena. MAGERIT prioriza, el pentesting verifica, la criptografía y los controles del OWASP Top 10 corrigen, y la auditoría sostiene la detección posterior.
3. El defecto más grave encontrado no fue criptográfico sino de **diseño de autorización**: confiar en datos del cliente. La criptografía no compensa un control de acceso ausente.

4. La construcción de una suite de ataques como especificación ejecutable **detectó dos errores en las propias pruebas**, lo que confirma que la verificación automatizada disciplinada —y no la revisión manual — es lo que sostiene la credibilidad de un informe de seguridad.
5. El laboratorio en contenedores elimina la barrera de infraestructura del curso: cualquier integrante, o el docente, reproduce el 100 % de los resultados en un equipo de 8 GB de RAM sin instalar herramientas de seguridad.
6. La doble versión sobre un mismo código resultó ser el recurso didáctico más potente del trabajo: permite ver el defecto, la mitigación y el resultado de la prueba en el mismo archivo y en la misma sesión.

11. Recomendaciones y trabajo futuro

1. **Política de contraseñas y bloqueo progresivo.** El riesgo residual de R4 exige longitud mínima, verificación contra listas de contraseñas comprometidas y bloqueo con retroceso exponencial.
2. **Anti-repetición en TOTP.** Almacenar el último paso aceptado por usuario para impedir la reutilización de un código dentro de la ventana de tolerancia.
3. **CSRF explícito.** `SameSite=Lax` mitiga, pero un token sincronizador en formularios es la defensa completa.
4. **Limitador distribuido.** El límite de intentos actual vive en memoria del proceso; en un despliegue con varias instancias debe residir en un almacén compartido (Redis).
5. **Análisis de archivos cargados.** Verificación de tipo real por contenido y análisis antimalware antes de aceptar un documento.
6. **Rotación de sesión y de claves.** Rotar el identificador de sesión al autenticarse y al cambiar de rol; planificar la rotación de la clave maestra de documentos.
7. **Centralización de la auditoría.** Enviar `audit_log` a un colector con retención, correlación y alertas (por ejemplo Loki + Grafana o un SIEM), y definir reglas de detección sobre los eventos que esta suite ya genera.
8. **Integración continua de la seguridad.** Ejecutar `attacks/run-all.sh` en cada cambio del código como puerta de calidad, y añadir análisis de dependencias (`cargo audit`) al flujo.
9. **Firma digital de documentos.** Incorporar firmas con clave pública para garantizar autenticidad y no repudio del contenido almacenado.

Referencias

1. OWASP Foundation. *OWASP Top 10 — 2021*. https://owasp.org/Top10/
2. OWASP Foundation. *Application Security Verification Standard (ASVS) 4.0.3*. https://owasp.org/www-project-application-security-verification-standard/
3. Ministerio de Hacienda y Administraciones Públicas (España). *MAGERIT v3 — Metodología de Análisis y Gestión de Riesgos de los Sistemas de Información*.
4. ISO/IEC 27005:2022. *Information security, cybersecurity and privacy protection — Guidance on managing information security risks*.
5. M'Raihi, D. et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF, 2011.
6. M'Raihi, D. et al. *HOTP: An HMAC-Based One-Time Password Algorithm*. RFC 4226, IETF, 2005.
7. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, IETF, 2021.
8. Grassi, P. A. et al. *Digital Identity Guidelines: Authentication and Lifecycle Management*. NIST SP 800-63B, 2017.
9. NIST. *Transitioning the Use of Cryptographic Algorithms and Key Lengths*. SP 800-131A Rev. 2, 2019.
10. Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.
11. Congreso de la República de Colombia. *Ley 1273 de 2009* — delitos informáticos.
12. Rust Project. *The Rust Programming Language*; documentación de `axum`, `sqlx`, `argon2`, `aes-gcm`, `hmac`.
13. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.

Anexos

Anexo A — Reproducción completa

```
git clone <repo> tcc-seguridad && cd tcc-seguridad
docker compose up -d --build            # base, app v1, app v2, gateway TLS y panel
bash attacks/run-all.sh                 # 18 ejecuciones → evidence/results.json
docker compose restart evidence         # refresca el panel en http://127.0.0.1:18090
```

Un ataque individual:

```
bash attacks/03-idor.sh http://127.0.0.1:18080 vulnerable   # exit 0: el defecto existe
bash attacks/03-idor.sh https://127.0.0.1:18443 hardened     # exit 1: bloqueado
```

Anexo B — Trazabilidad requisito → artefacto

Requisito de la actividad	Artefacto que lo cumple
Caso real o simulado	MiniGestor Documental (aplicación simulada, Rust)
Mínimo dos enfoques	Los cuatro: OWASP, criptografía, MAGERIT y ataques de red
Diagrama de arquitectura segura	`docs/diagramas/arquitectura.svg`
Flujo de datos e identificación de amenazas	`docs/diagramas/dfd-stride.svg` + tabla STRIDE (§6.2)
Propuesta de controles y contramedidas	§6.4 y `docs/riesgos/magerit.yml`
Mecanismos de cifrado	AES-256-GCM, Argon2id, HMAC-SHA256, TOTP, TLS
Autenticación	Login con Argon2id + segundo factor TOTP + sesión firmada
Firewall	`lab/firewall/lab.nft` (política de filtrado de referencia)
Pruebas de pentesting / seguridad web	`attacks/` (9 ataques automatizados)
Herramientas	Docker, curl, sqlmap/hydra/Burp/ZAP (compatibles), `tcpdump`/Wireshark, PostgreSQL
Documentación del proceso, pruebas y resultados	Este informe + `README.md` + panel de evidencias

Anexo C — Distribución de roles (CIPA de 3 integrantes)

Actividad	Integrante 1 (líder/riesgos)	Integrante 2 (desarrollo)	Integrante 3 (pentesting)
Análisis de riesgos MAGERIT	**R**	C	C
Arquitectura y diagramas	**R**	C	C
Modelado de amenazas STRIDE	**R**	C	C
Aplicación vulnerable (v1)	C	**R**	C
Aplicación endurecida (v2)	C	**R**	C
Suite de ataques	C	C	**R**
Capturas de tráfico y evidencias	I	C	**R**
Panel interactivo	I	C	**R**
Informe final	**R**	C	C

Actividad	Integrante 1 (líder/riesgos)	Integrante 2 (desarrollo)	Integrante 3 (pentesting)
Sustentación	C	C	**R**

R = responsable · C = colabora · I = informado